What is a K-Map?	**A Karnaugh Map (K-Map) is a graphical method used to simplify Boolean expressions. It reduces the number of variables and logic gates required in a digital circuit.**
Main purpose	**To obtain a simplified Boolean expression using grouping of adjacent 1s for SOP or adjacent 0s for POS.**
Basic rule	**Groups must contain 1, 2, 4, 8, 16... cells, i.e., powers of 2.**
Adjacency	**Adjacent cells differ in only one variable. The left and right edges are adjacent, and the top and bottom edges are also adjacent.**
Grouping rule	**Make the largest possible groups. Overlapping groups are allowed if required.**

Important Rules

1. Group only **1s** for SOP simplification.
2. Group only **0s** for POS simplification.
3. Groups must be in powers of 2: **1, 2, 4, 8, 16**.
4. Always form the **largest possible group**.
5. Every 1 should be covered at least once.
6. **Corner cells are adjacent**.
7. **Top and bottom cells are adjacent**.
8. **Overlapping groups are allowed** when they give a simpler expression.
9. Use **don't care conditions (X)** if they help form a larger group.

Steps to Solve a K-Map

1. **Identify the number of variables.**
2. **Draw the correct K-Map.**
3. **Enter 1s** according to the given minterms.
4. Make groups of **1, 2, 4, 8, or 16 cells**.
5. Select the **largest possible groups first**.
6. Find the variable values that remain constant in each group.
7. Write the simplified Boolean expression.
8. Combine all group expressions.

K-Map = A graphical technique for simplifying Boolean expressions.

- **2 variables → 4 cells**
- **3 variables → 8 cells**
- **4 variables → 16 cells**

Group sizes:

1, 2, 4, 8, 16

Main aim: To reduce the Boolean expression and minimize the required logic gates

CORNELL NOTES Topic: KARNAUGH MAP (K-MAP)

CUES / KEY POINTS

- **What is K-Map?**
  A Karnaugh Map (K-Map) is a graphical method used to simplify Boolean expressions.
- **Main Purpose**
  To obtain a simplified Boolean expression using grouping of adjacent 1s (SOP) or 0s (POS).
- **Basic Rule**
  Groups must contain 1, 2, 4, 8, 16... cells (powers of 2).
- **Adjacency**
  Adjacent cells differ in only one variable.
  Left & right edges are adjacent.
  Top & bottom edges are adjacent.
- **Grouping Rule**
  Make the largest possible groups.
  Overlapping groups are allowed if required.
- **Group Size vs Variables Eliminated**

Group Size (cells)	Variables Eliminated
1	0
2	1
4	2
8	3
16	4

- **Memory Tip**
  "Bigger the group, simpler the Boolean expression."

NOTES

1. 2-VARIABLE K-MAP

Variables: A and B

A\B	0	1
0	m_0	m_1
1	m_2	m_3

Minterms:
$m_0 = A'B'$
$m_1 = A'B$
$m_2 = AB'$
$m_3 = AB$

Example:
$F(A,B) = \Sigma m(0,1)$

A\B	0	1
0	1	1
1	0	0

Grouping:
Two 1s form a group of 2.
A = 0 (A')
B changes.

Simplified Expression: $F = A'$

2. 3-VARIABLE K-MAP

Variables: A, B and C
($2^3 = 8$ cells)

Gray code order ↓

A\BC	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

Example:
$F(A,B,C) = \Sigma m(1,3,5,7)$

A\BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

Grouping:
All four 1s form a group of 4.
C = 1 remains constant.

Simplified Expression: $F = C$

3. 4-VARIABLE K-MAP

Variables: A, B, C and D
($2^4 = 16$ cells)

Gray code order ↓

AB\CD	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

Example:
$F(A,B,C,D) = \Sigma m(0,1,2,3)$

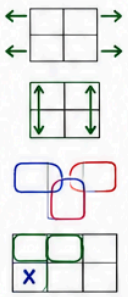
AB\CD	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

Grouping:
Four 1s form a group of 4.
A = 0 (A')
B = 0 (B')
C, D change.

Simplified Expression: $F = A'B'$

K-MAP GROUPING RULES

1. Group only 1s for SOP simplification.
2. Group only 0s for POS simplification.
3. Groups must be in powers of 2: 1, 2, 4, 8, 16.
4. Make the largest possible groups.
5. Every 1 (or 0) should be covered at least once.
6. Corner cells are adjacent.
7. Top & bottom cells are adjacent.
8. Overlapping groups are allowed.
9. Use don't care conditions (X) to form larger groups if possible.



STEPS TO SOLVE A K-MAP

1. Identify the number of variables.
2. Draw the correct K-Map.
3. Enter 1s according to the given minterms.
4. Make groups of 1, 2, 4, 8, or 16 cells.
5. Select the largest possible groups first.
6. Find the variable values that remain constant in each group.
7. Write the simplified Boolean expression.
8. Combine all group expressions.

K-Map = A graphical technique for simplifying Boolean expressions and minimizing logic gates.

CORNELL NOTES Topic: HALF ADDER

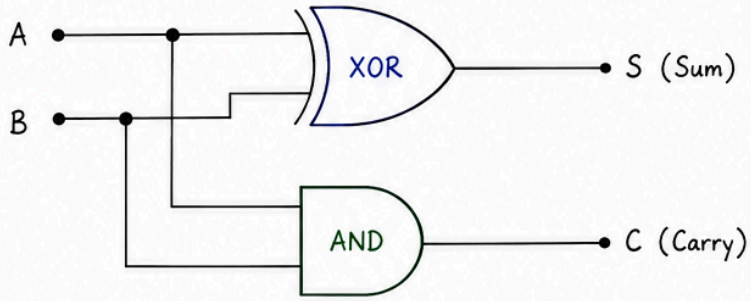
CUES / KEY POINTS

- **What is Half Adder?**
  A Half Adder is a combinational logic circuit that adds two 1-bit binary numbers.
- **Inputs and Outputs**
  Inputs : A, B
  Outputs :
  Sum (S)
  Carry (C)
- **Main Gates Used**
  XOR gate
  AND gate
- **Boolean Expressions**
  $S = A \oplus B$
  $C = A \cdot B$
  ($\oplus$ = XOR, $\cdot$ = AND)
- **Truth Table**
  Shows all possible combinations of inputs and their corresponding outputs.
- **Key Points**
  1. Adds two 1-bit numbers.
  2. Does not consider carry input from previous stage.
  3. Used as a building block for Full Adder.
- **Applications**
  Used in ALU, calculators, adders, microprocessors, etc.
- **Memory Tip**
  XOR → Sum
  AND → Carry

NOTES

Half Adder is used to add two 1-bit binary numbers and gives the Sum and Carry as output.

1. Logic Circuit



2. Boolean Expressions

$S = A \oplus B$

$C = A \cdot B$

3. Truth Table

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

4. Explanation of Truth Table

→ A = 0, B = 0 → 0 + 0 = 0 → S = 0, C = 0

→ A = 0, B = 1 → 0 + 1 = 1 → S = 1, C = 0

→ A = 1, B = 0 → 1 + 0 = 1 → S = 1, C = 0

→ A = 1, B = 1 → 1 + 1 = 10 → S = 0, C = 1

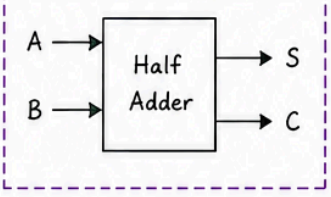
5. Example

Example: Add A = 1 and B = 1

Binary :
$$\begin{array}{r} 1 \\ +\ 1 \\ \hline 10 \end{array}$$

⇒ Sum (S) = 0, Carry (C) = 1

Logic Symbol



Summary

- Half Adder is a combinational circuit.
- It adds two 1-bit inputs and produces Sum and Carry.
- It uses one XOR gate (for Sum) and one AND gate (for Carry).
- It is the basic building block for Full Adder and other arithmetic circuits.

CORNELL NOTES Topic: FULL ADDER

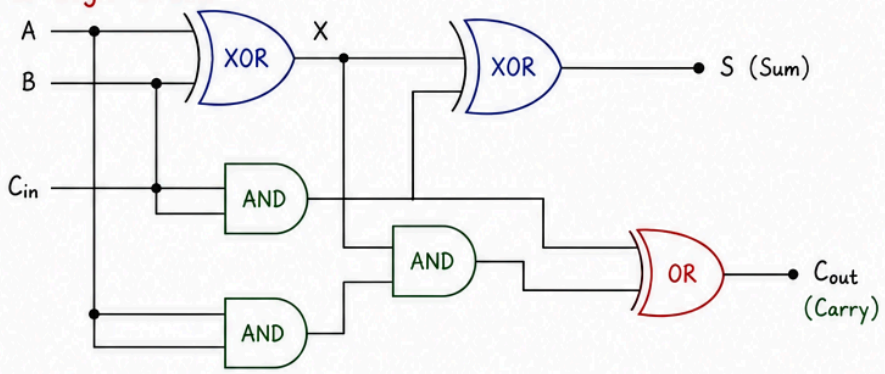
CUES / KEY POINTS

- **What is Full Adder?**
  A Full Adder is a combinational logic circuit that adds three 1-bit binary numbers.
- **Inputs and Outputs**
  Inputs : A, B, C_{in}
  Outputs :
  Sum (S)
  Carry (C_{out})
- **Main Gates Used**
  Two XOR gates,
  Two AND gates,
  One OR gate
- **Boolean Expressions**
  $S = A \oplus B \oplus C_{in}$
  $C_{out} = AB + BC_{in} + AC_{in}$
  ($\oplus$ = XOR, + = OR, $\cdot$ = AND)
- **Truth Table**
  Shows all possible combinations of inputs and their corresponding outputs.
- **Key Points**
  1. Adds three 1-bit numbers.
  2. Considers carry input from previous stage.
  3. Used to build Ripple Carry Adder.
- **Applications**
  Used in ALU, calculators, adders, microprocessors, etc.
- **Memory Tip**
  Two XOR → Sum
  Two AND + One OR → Carry

NOTES

Full Adder is used to add three 1-bit binary numbers: A, B and C_{in} and produces Sum (S) and Carry out (C_{out}).

1. Logic Circuit



2. Boolean Expressions

$$\text{Sum (S)} = A \oplus B \oplus C_{in}$$

$$\text{Carry out } (C_{out}) = AB + BC_{in} + AC_{in}$$

Where, $\oplus$ = XOR, + = OR, $\cdot$ = AND

3. Truth Table

A	B	C_{in}	Sum (S)	Carry out (C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4. Explanation of Truth Table

- 0 + 0 + 0 = 0 → S = 0, C_{out} = 0
- 0 + 0 + 1 = 1 → S = 1, C_{out} = 0
- 0 + 1 + 0 = 1 → S = 1, C_{out} = 0
- 0 + 1 + 1 = 2 (10_2) → S = 0, C_{out} = 1
- 1 + 0 + 0 = 1 → S = 1, C_{out} = 0
- 1 + 0 + 1 = 2 (10_2) → S = 0, C_{out} = 1
- 1 + 1 + 0 = 2 (10_2) → S = 0, C_{out} = 1
- 1 + 1 + 1 = 3 (11_2) → S = 1, C_{out} = 1

5. Example

Example: Add A = 1, B = 1, C_{in} = 1

```
  A      1
  B    + 1
  Cin  + 1
  ------
    11  (in binary)
```

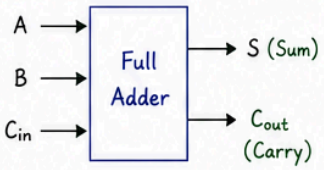
Therefore, Sum (S) = 1
Carry out (C_{out}) = 1

From Truth Table (or Expression)

$S = A \oplus B \oplus C_{in}$
$= 1 \oplus 1 \oplus 1$
$= 1$

$C_{out} = AB + BC_{in} + AC_{in}$
$= 1.1 + 1.1 + 1.1$
$= 1 + 1 + 1 = 1$

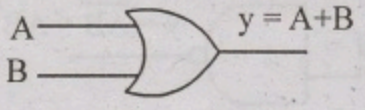
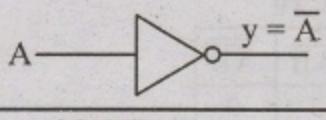
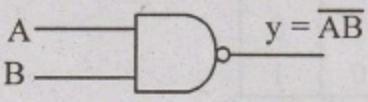
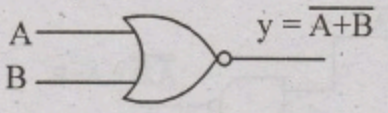
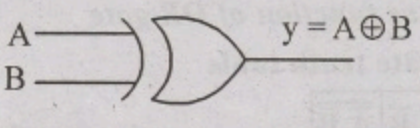
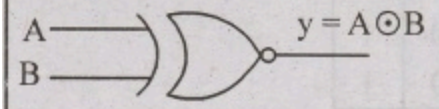
6. Logic Symbol



Summary

- Full Adder adds three 1-bit inputs (A, B, C_{in}) and produces Sum and Carry out.
- It uses 2 XOR gates to compute Sum and 2 AND gates + 1 OR gate to compute Carry out.
- It is the basic building block for multi-bit adders like Ripple Carry Adder.

Table 4.9 Logical gates and truth table

Logical gate	Truth table
AND gate 	A B y=AB 0 0 0 0 1 0 1 0 0 1 1 1
OR gate 	A B y=A+B 0 0 0 0 1 1 1 0 1 1 1 1
NOT gate 	A y=$\overline{A}$ 0 1 1 0
NAND gate 	A B y=$\overline{AB}$ 0 0 1 0 1 1 1 0 1 1 1 0
NOR gate 	A B y=$\overline{A+B}$ 0 0 1 0 1 0 1 0 0 1 1 0
EX–OR gate 	A B y=A⊕B 0 0 0 0 1 1 1 0 1 1 1 0
EX–NOR gate 	A B y=A⊙B 0 0 1 0 1 0 1 0 0 1 1 1

BASIC AND LOGIC GATES

Symbol, Truth Table and Boolean Equation

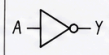
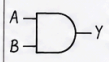
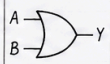
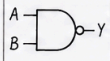
Date : 16/05/2025

Page : 1

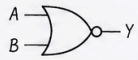
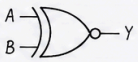
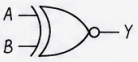
CUE COLUMN

- → What are basic gates?
- → What are logic gates?
- → How many types of gates are used?
- → What is the symbol of each gate?
- → What is the truth table?
- → What is the Boolean equation of each gate?
- → Why are these gates important?

BASIC GATES

No.	Gate Name	Symbol	Truth Table	Boolean Equation
1.	BUFFER (YES Gate)		A | Y 0 | 0 1 | 1	$Y = A$
2.	NOT (INVERTER)		A | Y 0 | 1 1 | 0	$Y = \overline{A}$
3.	AND		A B | Y 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1	$Y = A \cdot B$ $= AB$
4.	OR		A B | Y 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1	$Y = A + B$
5.	NAND		A B | Y 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0	$Y = \overline{A \cdot B}$ $= \overline{AB}$

LOGIC GATES

No.	Gate Name	Symbol	Truth Table	Boolean Equation
6.	NOR		A B | Y 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0	$Y = \overline{A + B}$
7.	XOR (Exclusive OR)		A B | Y 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0	$Y = A \oplus B$ $= \overline{A}B + A\overline{B}$
8.	XNOR (Exclusive NOR)		A B | Y 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1	$Y = \overline{A \oplus B}$ $= AB + \overline{A}\overline{B}$
9.	IMPLICATION (If A then B)		A B | Y 0 0 | 1 0 1 | 1 1 0 | 0 1 1 | 1	$Y = \overline{A} + B$
10.	BICONDIONAL (Equivalence)		A B | Y 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1	$Y = A \equiv B$ $= AB + \overline{A}\overline{B}$

SUMMARY COLUMN

(Write key points in short) →

SUMMARY (Bottom)

- Basic gates are the fundamental building blocks: BUFFER, NOT, AND, OR.
- Other gates (NAND, NOR, XOR, etc.) are derived from basic gates.
- These gates are used to design digital circuits and perform logical operations.

CORNELL NOTES

ASSOCIATIVE AND COMMUTATIVE LAWS
(Boolean Algebra)

Date : 16/05/2025
Page : 1

CUE COLUMN

* What is Commutative law ?
* Commutative law for OR ?
* Commutative law for AND ?
---
* What is Associative law ?
* Associative law for OR ?
* Associative law for AND ?
* Why are these laws important ?

MAIN NOTES

1. COMMUTATIVE LAW

Changing the order of inputs does not change the output.

(a) For OR operation

Statement :

$$A + B = B + A$$

A	B	A + B	B + A
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Proof :
Since $A+B$ and $B+A$ are always equal, the law is proved. ✓

(b) For AND operation

Statement :

$$A \cdot B = B \cdot A$$

A	B	A · B	B · A
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Proof :
Since $A \cdot B$ and $B \cdot A$ are always equal, the law is proved. ✓

2. ASSOCIATIVE LAW

Changing the grouping (brackets) of variables does not change the output.

(a) For OR operation

Statement :

$$(A + B) + C = A + (B + C)$$

A	B	C	(A+B)+C	A+(B+C)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Proof :
Both columns are identical. Hence, $(A+B)+C = A+(B+C)$ Proved. ✓

(b) For AND operation

Statement :

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

A	B	C	(A·B)·C	A·(B·C)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Proof :
Both columns are identical. Hence, $(A \cdot B) \cdot C = A \cdot (B \cdot C)$ Proved. ✓

SUMMARY COLUMN

(Write key points in short) →

SUMMARY (BOTTOM)

* Commutative Law : Changing the order of variables does not change the output.
  - OR : $A + B = B + A$ | - AND : $A \cdot B = B \cdot A$
* Associative Law : Changing the grouping of variables does not change the output.
  - OR : $(A + B) + C = A + (B + C)$ | - AND : $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
* These laws reduce the complexity of Boolean expressions and simplify digital circuit design.

NAND as Universal Gate

(Any Logic Function can be Implemented using NAND Gate)

Date : 16/05/2025

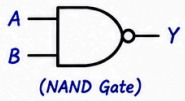
Page : 1

CUE COLUMN

- What is Universal Gate ?
- NAND gate symbol and truth table
- How NAND acts as NOT, AND, OR
- Why NAND is called Universal Gate ?

Main Notes

1. NAND Gate Symbol and Truth Table



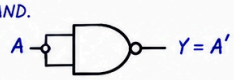
(NAND Gate)

A	B	$Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

2. NAND as NOT Gate

Tie both inputs of NAND.

$Y = (A \cdot A)' = A'$

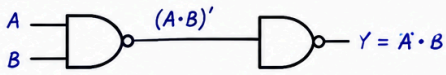


→ NAND works as an Inverter (NOT gate).

3. NAND as AND Gate

Use NAND followed by NAND as inverter.

$Y = ((A \cdot B)')' = A \cdot B$

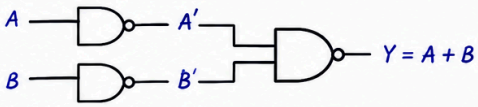


→ Hence NAND can implement AND.

4. NAND as OR Gate

Use De Morgan's law:

$A + B = (A' \cdot B')'$



→ Hence NAND can implement OR.

5. Why NAND is called Universal Gate ?

- → A logic gate is called **universal gate** if any other basic logic function (NOT, AND, OR) can be implemented using only that gate.
- → NAND can perform NOT, AND and OR operations using itself only.
- → Therefore, NAND is a universal gate.

SUMMARY COLUMN

(Write key points in short) →

- NAND = Universal Gate (implements NOT, AND, OR).
- NOT using NAND : Connect both inputs of NAND.
- AND using NAND : NAND followed by NAND (inverter).
- OR using NAND : Invert inputs, then NAND the results.

CORNELL NOTES

DE MORGAN'S LAWS

(First and Second Law with Truth Table)

Date : 16/05/2025

Page : 1

CUE COLUMN

→ What are De Morgan's Laws ?

→ What is First Law ?

→ Truth table for First Law ?

→ What is Second Law ?

→ Truth table for Second Law ?

→ How to remember De Morgan's Laws ?

→ Why are these laws important ?

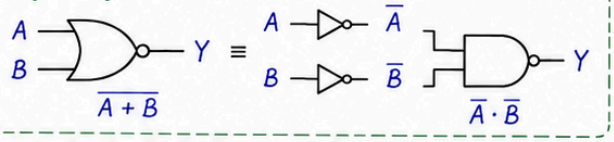
MAIN NOTES

1. De Morgan's First Law

The complement of OR is equal to the AND of the complements.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Logic Diagram :



Truth Table :

A	B	A + B	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

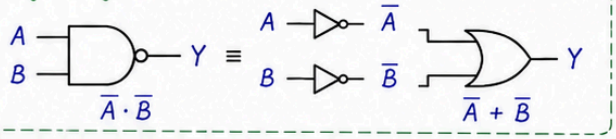
Since $\overline{A+B}$ and $\overline{A} \cdot \overline{B}$ columns are identical.
Hence, First Law is proved. ✓

2. De Morgan's Second Law

The complement of AND is equal to the OR of the complements.

$$\overline{A} \cdot \overline{B} = \overline{A} + \overline{B}$$

Logic Diagram :



Truth Table :

A	B	A · B	$\overline{A} \cdot \overline{B}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Since $\overline{A} \cdot \overline{B}$ and $\overline{A} + \overline{B}$ columns are identical.
Hence, Second Law is proved. ✓

SUMMARY COLUMN

(Write key points in short) →

SUMMARY (BOTTOM)

- First Law : $\overline{A + B} = \overline{A} \cdot \overline{B}$ (OR → AND, complement each variable)
- Second Law : $\overline{A} \cdot \overline{B} = \overline{A} + \overline{B}$ (AND → OR, complement each variable)
- Shortcut : Bar outside → Change the operator (OR ↔ AND) and put bar on every variable.
- These laws are useful in simplifying Boolean expressions and designing logic circuits.